

✓ Movie Revenue Prediction

Yetro Cheng, Fardin Iqbal, Tanjim Ahammad

Movie Revenue Prediction**

Film industry is booming, the revenues are growing. There are many factors which affect the revenue of a film. In this project, you will explore what features can help to predict the revenue.

Datasets:

The "movie.zip" file contains the datasets to be used for this project and a file describing the various columns in the data. You must split the dataset yourself into training, testing, and cross validation data(when required). Data points provided include cast, crew, plot keywords, budget, posters, release dates,, languages, production companies, and countries.

EDA (10 points):

Get familiar with the dataset and decide what features and observations will be useful. Make good use of visualizations.

Specific tasks may include but are not limited to:

- Clean the dataset, remove the outliers, before any data analysis. Explain what you did.
- Some of the columns contain lists and dictionaries. Extract information you need and reformat them.
- Count the number of movies released by day of week, month and year, are there any patterns that you observe?
- What are the movie genre trend shifting patterns that you can observe from the dataset?
- What are the strongest and weakest features correlated with movie revenue?
- You can also use some external datasets to integrate into your revenue prediction analysis to make it better.

Modeling and Question Answering (10 points):

Extract the features you think are necessary in predicting the movie revenue.

Build three models, train them on the training set, and predict the revenue on the test set (after dropping the revenue column in the test set). Explain how each model works (briefly introduce the machine learning algorithms behind them). Evaluate the performance of each model based on the original outcome in the test set. If your predictions are not so accurate, what do you think is the reason? Report your accuracy using metrics such as Residual Standard Error (RSE). Split the data further to include a cross validation set. Did this improve your model's performance on the test set?

Project Report (10 points):

You are required to document your project, which can be included in the notebook itself. Don't forget to include the team members contribution information in the documentation. Include visualizations to prove your point. You should prepare a powerpoint presentation, which can help you during the demo.

Demo (5 points):

Sign up for a Zoom session with the mentor to present your project. All the team members should be present during the demo. Be prepared to answer questions related to your work. You should present your findings for the project, and you should also be able to run your code.

Submission:

Submit the following on Blackboard:

1. Code in pdf and ipynb format
2. Project Presentation in Powerpoint format

✓ Dataset Upload

```
# Upload dataset files
from google.colab import files
import pandas as pd

# Load datasets
movies_df = pd.read_csv("tmdb_5000_movies.csv")
credits_df = pd.read_csv("tmdb_5000_credits.csv")
```

✓ Data cleaning (Tanjim) -

```
import pandas as pd

# Store original count
original_movie_count = len(movies_df)

# --- COMPOUND FILTERS ---
# Budget vs Revenue outliers
finance_filter = (
    (movies_df['budget'] <= 1.75e8) &
    (movies_df['revenue'] <= 0.7e9)
)

# Vote Count vs Vote Average outliers
vote_filter = (
    (movies_df['vote_count'] <= 8000) &
    (movies_df['vote_average'] >= 3.5) &
    (movies_df['vote_average'] <= 8.3)
)

# --- INDIVIDUAL FILTERS ---
runtime_filter = (movies_df['runtime'].notna()) & (movies_df['runtime'] >= 60) & (movies_df['runtime'] <= 200)
popularity_filter = (movies_df['popularity'] <= 150)

# Combine all filters (must pass all)
combined_filter = finance_filter & vote_filter & runtime_filter & popularity_filter

# Apply filtering
cleaned_movies_df = movies_df[combined_filter].copy()
removed_movie_count = original_movie_count - len(cleaned_movies_df)

# Filter corresponding credits
valid_movie_ids = set(cleaned_movies_df['id'])
cleaned_credits_df = credits_df[credits_df['movie_id'].isin(valid_movie_ids)].copy()

# Reset index
cleaned_movies_df.reset_index(drop=True, inplace=True)
cleaned_credits_df.reset_index(drop=True, inplace=True)

# Report
print("Original number of movies:", original_movie_count)
print("Movies remaining after composite filter:", len(cleaned_movies_df))
print("Movies removed:", removed_movie_count)
print("Credits remaining:", len(cleaned_credits_df))
```

Original number of movies: 4803
 Movies remaining after composite filter: 4504
 Movies removed: 299
 Credits remaining: 4504

```
import numpy as np

# Compute and print summary statistics for revenue
mean_revenue = cleaned_movies_df['revenue'].mean()
median_revenue = cleaned_movies_df['revenue'].median()
std_revenue = cleaned_movies_df['revenue'].std()

print(f"Mean Revenue: ${mean_revenue:,.2f}")
print(f"Median Revenue: ${median_revenue:,.2f}")
print(f"Standard Deviation of Revenue: ${std_revenue:,.2f}")
```

Mean Revenue: \$66,495,724.18
 Median Revenue: \$19,475,081.50
 Standard Deviation of Revenue: \$107,108,987.41

Documentation (Data Cleaning) -

Before any feature engineering or modeling, it was essential to clean the dataset to remove anomalies, missing values, and outliers that could bias the results. The original dataset included movies with unrealistic or extreme values for several important features, such as budget ,

revenue, vote_count, and runtime. These can distort model training and lead to overfitting on outlier points. The following filters were applied:

Compound Filters:

1. Budget and Revenue Filter (finance_filter)

We removed movies with a budget exceeding 175 million (USD) and revenue exceeding 700 million (USD). Extremely large values in these columns often correspond to blockbuster films with different financial dynamics compared to mid-range films, and may dominate the training loss if left in.

2. Vote Count and Vote Average Filter (vote_filter)

To reduce the influence of overly popular or unpopular movies, we kept only those with a vote count under 8000 and an average vote between 3.5 and 8.3. This ensures we focus on relatively mainstream films while excluding extreme cases with unusual viewer engagement.

Individual Filters:

3. Runtime Filter (runtime_filter)

We removed entries with missing runtime values, and only included movies with a runtime between 60 and 200 minutes. This excludes shorts and excessively long films, which are likely to be outliers.

4. Popularity Filter (popularity_filter)

Entries with a popularity score over 150 were excluded. These tend to reflect unusual data behavior, viral re-releases, or legacy metadata that can mislead the learning process.

Application of Filters:

All filters were applied together using a logical AND condition, meaning a movie had to satisfy **all** criteria to be included in the cleaned dataset. This resulted in the removal of a substantial number of outlier entries, reducing the dataset to a more statistically reliable and representative subset for prediction tasks.

The same filtering was applied to the credits dataset by matching on movie_id, ensuring consistency between the two tables.

This cleaned dataset provides a sound foundation for downstream feature engineering and model training.

✓ Feature Engineering (Yetto) -

```
import pandas as pd
import numpy as np
import ast
from collections import Counter

# Merge datasets
df = pd.merge(cleaned_movies_df, cleaned_credits_df, left_on='id', right_on='movie_id')

# --- Genres ---
def extract_genres(x):
    try:
        return [d['name'] for d in ast.literal_eval(x)]
    except:
        return []

df['genre_list'] = df['genres'].apply(extract_genres)
all_genres = [g for sublist in df['genre_list'] for g in sublist]
common_genres = set(pd.Series(all_genres).value_counts()[lambda x: x >= 50].index)

for genre in common_genres:
    df[f'genre_{genre}'] = df['genre_list'].apply(lambda x: int(genre in x))

df['genre_Other'] = df['genre_list'].apply(lambda x: int(any(g not in common_genres for g in x)))

# --- Budget and Popularity (log) ---
df['log_budget'] = np.log1p(df['budget'])
df['log_popularity'] = np.log1p(df['popularity'])

# --- Budget Buckets ---
def budget_bucket(b):
    if b < 2e7: return 'low'
    elif b < 1e8: return 'mid'
    else: return 'high'

df['budget_bucket'] = df['budget'].apply(budget_bucket)
```

```

df = pd.concat([df, pd.get_dummies(df['budget_bucket'], prefix='budget']], axis=1)
df.drop(columns=['budget_bucket'], inplace=True)

# --- Runtime Buckets ---
def runtime_bucket(rt):
    if rt < 90: return 'short'
    elif rt <= 130: return 'medium'
    else: return 'long'

df['runtime_bucket'] = df['runtime'].apply(runtime_bucket)
df = pd.concat([df, pd.get_dummies(df['runtime_bucket'], prefix='runtime']], axis=1)

# --- Release Period (5-year bins) ---
df['release_date'] = pd.to_datetime(df['release_date'], errors='coerce')
df['release_year'] = df['release_date'].dt.year

# Add month and day of week
df['release_month'] = df['release_date'].dt.month
df['release_day_of_week'] = df['release_date'].dt.dayofweek # 0 = Monday, 6 = Sunday

# Add holiday period flags
df['is_summer_release'] = df['release_month'].isin([5, 6, 7, 8]).astype(int)
df['is_holiday_release'] = df['release_month'].isin([11, 12]).astype(int)
df['is_spring_break'] = df['release_month'].isin([3, 4]).astype(int)

# One-hot encode months
df = pd.concat([df, pd.get_dummies(df['release_month'], prefix='month']], axis=1)

def five_year_bin(y):
    if pd.isna(y): return 'Unknown'
    elif y < 1980: return 'Before 1980'
    else: return f"{int(y//5*5)}-{int(y//5*5)+4}"

df['release_5yr'] = df['release_year'].apply(five_year_bin)
df = pd.concat([df, pd.get_dummies(df['release_5yr'], prefix='period']], axis=1)

# --- Production Company Features ---
def extract_production_companies(x):
    try:
        return [d['name'] for d in ast.literal_eval(x)]
    except:
        return []

df['production_company_list'] = df['production_companies'].apply(extract_production_companies)

# Define major studios
major_studios = [
    'Warner Bros', 'Warner Bros.', 'Universal Pictures', 'Walt Disney', 'Disney',
    'Columbia Pictures', 'Paramount', 'Paramount Pictures', '20th Century Fox',
    'New Line Cinema', 'Sony Pictures', 'MGM', 'Lionsgate', 'DreamWorks'
]

# Create flags for major studios
for studio in major_studios:
    df[f'studio_{studio.replace(" ", "_")}'] = df['production_company_list'].apply(
        lambda x: int(any(studio in company for company in x))
    )

# Simplify to a general "major studio" flag
df['is_major_studio'] = df['production_company_list'].apply(
    lambda x: int(any(any(studio in company for studio in major_studios) for company in x))
)

# Calculate average revenue per studio
studio_avg_revenue = {}
for i, row in df.iterrows():
    for company in row['production_company_list']:
        if company not in studio_avg_revenue:
            studio_avg_revenue[company] = {'total_revenue': 0, 'count': 0}
        studio_avg_revenue[company]['total_revenue'] += row['revenue']
        studio_avg_revenue[company]['count'] += 1

# Convert to average
for company in studio_avg_revenue:
    if studio_avg_revenue[company]['count'] > 0:
        studio_avg_revenue[company] = studio_avg_revenue[company]['total_revenue'] / studio_avg_revenue[company]['count']
    else:

```

```

studio_avg_revenue[company] = 0

# Add studio average revenue feature
df['studio_avg_revenue'] = df['production_company_list'].apply(
    lambda companies: max([studio_avg_revenue.get(company, 0) for company in companies]) if companies else 0
)

# --- Franchise ---
if 'belongs_to_collection' in df.columns:
    df['is_franchise'] = df['belongs_to_collection'].apply(lambda x: int(pd.notna(x) and x != 'null'))

# Extract collection name
def extract_collection(x):
    try:
        if pd.notna(x) and x != 'null':
            return ast.literal_eval(x)['name']
        return None
    except:
        return None

df['collection_name'] = df['belongs_to_collection'].apply(extract_collection)

# Calculate average revenue per collection
collection_avg_revenue = {}
for collection, group in df.groupby('collection_name'):
    if collection and pd.notna(collection):
        collection_avg_revenue[collection] = group['revenue'].mean()

# Add collection average revenue feature
df['franchise_avg_revenue'] = df['collection_name'].apply(
    lambda x: collection_avg_revenue.get(x, 0) if x and pd.notna(x) else 0
)
else:
    df['is_franchise'] = 0
    df['franchise_avg_revenue'] = 0

# --- Cast and Director (high-impact only) ---
def extract_director(crew_str):
    try:
        for member in ast.literal_eval(crew_str):
            if member.get('job') == 'Director':
                return member.get('name')
    except:
        return None

df['director'] = df['crew'].apply(extract_director)
df['vote_weighted'] = df['vote_average'] * np.log1p(df['vote_count'])

# Top directors: at least 5 high-impact films
high_impact = df[(df['budget'] > 5e7) & (df['vote_weighted'] > df['vote_weighted'].quantile(0.75))]
top_directors = set(high_impact['director'].value_counts()[lambda x: x >= 5].index)
df['is_famous_director'] = df['director'].apply(lambda x: int(x in top_directors))

# Calculate average revenue per director
director_avg_revenue = {}
for director, group in df.groupby('director'):
    if director and pd.notna(director):
        director_avg_revenue[director] = group['revenue'].mean()

# Add director's average revenue feature
df['director_avg_revenue'] = df['director'].apply(
    lambda x: director_avg_revenue.get(x, 0) if x and pd.notna(x) else 0
)

# Top actors: at least 5 appearances in high-impact films
def get_actor_names(cast_str):
    try:
        return [d['name'] for d in ast.literal_eval(cast_str)]
    except:
        return []

df['cast_names'] = df['cast'].apply(get_actor_names)
actor_counter = Counter()
for names in high_impact['cast'].apply(get_actor_names):
    actor_counter.update(names)
top_actors = set([name for name, count in actor_counter.items() if count >= 5])
df['has_famous_actor'] = df['cast_names'].apply(lambda names: int(any(n in top_actors for n in names)))

```

```
# Add cast density feature - number of famous actors per film
df['famous_actor_count'] = df['cast_names'].apply(lambda names: sum(1 for n in names if n in top_actors))

# --- Language simplified ---
df['lang_non_en'] = (df['original_language'] != 'en').astype(int)

# --- Create interaction terms between important features ---
df['budget_x_popularity'] = df['log_budget'] * df['popularity']
df['budget_x_runtime'] = df['log_budget'] * df['runtime']
df['budget_x_vote_weighted'] = df['log_budget'] * df['vote_weighted']
df['franchise_x_budget'] = df['is_franchise'] * df['log_budget']
df['famous_director_x_budget'] = df['is_famous_director'] * df['log_budget']
df['famous_actor_x_budget'] = df['has_famous_actor'] * df['log_budget']

# More complex interaction terms
df['holiday_family_film'] = df['is_holiday_release'] * df['genre_Family']
df['summer_action_film'] = df['is_summer_release'] * df['genre_Action']
df['franchise_famous_actor'] = df['is_franchise'] * df['famous_actor_count']
df['franchise_famous_director'] = df['is_franchise'] * df['is_famous_director']
df['major_studio_budget'] = df['is_major_studio'] * df['log_budget']
df['weekend_release'] = (df['release_day_of_week'] >= 4).astype(int)
df['weekend_summer_release'] = df['weekend_release'] * df['is_summer_release']

# --- Add polynomial features for key numeric predictors ---
df['log_budget_squared'] = df['log_budget'] ** 2
df['popularity_squared'] = df['popularity'] ** 2
df['vote_weighted_squared'] = df['vote_weighted'] ** 2
df['runtime_squared'] = df['runtime'] ** 2

# --- Drop unused ---
df.drop(columns=[
    'genre_list', 'cast_names', 'has_homepage', 'num_cast', 'vote_average', 'vote_count',
    'num_production_companies', 'popularity_per_year', 'budget_per_year',
    'runtime_bucket', 'release_month', 'season', 'belongs_to_collection',
    'crew', 'cast', 'genres', 'homepage', 'original_language', 'overview',
    'production_companies', 'production_countries', 'spoken_languages',
    'tagline', 'title', 'keywords', 'release_5yr', 'collection_name', 'production_company_list'
], errors='ignore', inplace=True)

# Assume missing release day is Friday (day 4)
df['release_day_of_week'] = df['release_day_of_week'].fillna(4)

# Impute missing runtime with mean
df['runtime'] = df['runtime'].fillna(df['runtime'].mean())

# Recompute dependent columns based on imputed runtime
df['budget_x_runtime'] = df['log_budget'] * df['runtime']
df['runtime_squared'] = df['runtime'] ** 2

# Ready for modeling in df
```

Documentation (Feature Engineering) -

To enhance model performance, we engineered a broad set of features derived from domain knowledge and insights from exploratory data analysis. These features were chosen because they are known or hypothesized to correlate strongly with a movie's box office success. The goal was to transform raw, nested, or textual data into structured, interpretable, and predictive features.

1. Genres

- **Why:** Genre significantly influences audience size and revenue potential (e.g., Action and Animation films tend to generate high returns).
- **How:** Parsed JSON from the `genres` column. Created binary features for genres that appear in at least 50 films. A catch-all `genre_Other` flag was added to capture rare genres.

2. Budget and Popularity (Log Transformed)

- **Why:** Higher budget often allows for better production, marketing, and cast, all of which boost revenue. Popularity is a proxy for online traction or anticipation.
- **How:** Log-transformed budget and popularity to mitigate skew and compress extreme values, improving model stability.

3. Budget Buckets

- **Why:** Revenue behavior changes across different budget levels (e.g., low-budget films can overperform proportionally; blockbusters tend to have high absolute revenue).
- **How:** Budget categorized into `low`, `mid`, and `high` tiers to capture threshold effects.

4. Runtime Buckets

- **Why:** Extremely short or long films tend to underperform due to limited showtimes or niche appeal. Mid-length films (90–130 minutes) are mainstream-friendly.
- **How:** Runtime was bucketed and one-hot encoded.

5. Temporal Release Features

- **Why:** Release timing affects competition and attendance. Summer and holiday periods are lucrative; weekends typically see higher foot traffic.
- **How:** Extracted `release_month`, `day_of_week`, and derived flags for `summer`, `holiday`, `spring break`, and `weekend releases`. Binned years into 5-year periods to capture generational trends.

6. Production Companies / Studios

- **Why:** Established studios often have more marketing power and distribution networks. Certain studios (e.g., Disney, Warner Bros) consistently release high-grossing films.
- **How:** Identified major studios with binary flags and computed `studio_avg_revenue` to quantify historical success per studio.

7. Franchise Membership

- **Why:** Sequels and franchise films benefit from brand recognition and built-in audiences, typically outperforming standalone films.
- **How:** Used `belongs_to_collection` to create `is_franchise` and computed `franchise_avg_revenue` as a proxy for brand momentum.

8. Director and Cast Prestige

- **Why:** High-profile directors and actors attract attention and investor confidence. Star power has a measurable effect on revenue.
- **How:**
 - Parsed directors from the `crew` column.
 - Identified "famous" directors and actors based on repeated presence in high-impact films (those with large budgets and high vote-weighted ratings).
 - Created flags and counts (`is_famous_director`, `famous_actor_count`, etc.).
 - Computed historical average revenue per director for additional signal.

9. Language

- **Why:** English-language films tend to have broader international release and visibility.
- **How:** Created a binary flag `lang_non_en` to indicate non-English language films.

10. Interaction Terms

- **Why:** Feature combinations can reveal richer patterns (e.g., high-budget + famous actor may be more impactful than either alone).
- **How:**
 - Created interaction terms like `budget_x_popularity`, `famous_director_x_budget`, `franchise_famous_actor`, and `major_studio_budget`.
 - Seasonal interactions (e.g., `holiday_family_film`) were included to account for genre-performance timing synergies.

11. Polynomial Features

- **Why:** To capture nonlinear relationships where the effect of a feature increases or decreases at a changing rate.
- **How:** Squared key numerical features: `log_budget`, `popularity`, `vote_weighted`, and `runtime`.

12. Final Cleanup and Imputation

- Dropped columns that were:
 - Redundant (e.g., original nested JSON strings)
 - Non-numeric or text-heavy
 - Inappropriate for predictive modeling (e.g., `homepage`, `overview`, `tagline`)
- Imputed missing `runtime` values with the mean.

- Assumed missing `release_day_of_week` to be Friday, the standard movie release day.

Conclusion:

This feature set balances domain expertise with empirical analysis. It captures both individual predictors and interaction effects, enabling our models to learn both direct and contextual revenue drivers. By incorporating financial, temporal, production, and cast-related variables, we maximize the model's ability to generalize across film types.

✓ Feature Selection/Definition -

```
from sklearn.model_selection import train_test_split

# Target variable
y = df['revenue']

# Ensure 'is_franchise' exists (in case feature engineering step skipped it)
if 'is_franchise' not in df.columns:
    df['is_franchise'] = 0

# --- Core numeric and binary features ---
base_features = [
    'log_budget', 'popularity', 'runtime',
    'is_famous_director', 'has_famous_actor',
    'vote_weighted', 'is_franchise',
    'director_avg_revenue', 'studio_avg_revenue', 'franchise_avg_revenue',
    'famous_actor_count', 'is_major_studio'
]

# --- Release date and seasonality features ---
seasonality_features = [
    'release_day_of_week', 'is_summer_release', 'is_holiday_release',
    'is_spring_break', 'weekend_release'
]

# Get month dummy features
month_features = [col for col in df.columns if col.startswith('month_')]

# --- Interaction and polynomial features ---
interaction_features = [
    'budget_x_popularity', 'budget_x_runtime', 'budget_x_vote_weighted',
    'franchise_x_budget', 'famous_director_x_budget', 'famous_actor_x_budget',
    'holiday_family_film', 'summer_action_film', 'franchise_famous_actor',
    'franchise_famous_director', 'major_studio_budget', 'weekend_summer_release'
]

polynomial_features = [
    'log_budget_squared', 'popularity_squared',
    'vote_weighted_squared', 'runtime_squared'
]

# --- Studio features ---
studio_features = [col for col in df.columns if col.startswith('studio_') and col != 'studio_avg_revenue']

# --- Feature groups from original code ---
runtime_features = [col for col in df.columns if col.startswith('runtime_')]
language_features = ['lang_non_en'] if 'lang_non_en' in df.columns else []
genre_features = [col for col in df.columns if col.startswith('genre_')]
season_features = [col for col in df.columns if col.startswith('season_')]
budget_bucket_features = [col for col in df.columns if col.startswith('budget_')]
period_features = [col for col in df.columns if col.startswith('period_')]

# Combine all features
feature_cols = (
    base_features +
    seasonality_features +
    month_features +
    studio_features +
    interaction_features +
    polynomial_features +
    runtime_features +
    language_features +
    genre_features +
    season_features +
    budget_bucket_features +
    period_features
)
```

```

    period_features
)

# Subset feature matrix
X = df[feature_cols]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

```

Documentation (Feature Selection and Definition) -

After engineering a wide array of features, we carefully selected subsets to include in model training. Our goal was to retain informative, interpretable, and independent features that could maximize the predictive power of our models without introducing excessive redundancy or noise.

We grouped features into thematic categories based on their relevance to the movie industry and potential correlation with revenue.

1. Core Numeric and Binary Features

- These include critical variables known to directly influence revenue:
 - `log_budget`: Higher production budgets generally lead to wider releases and better quality.
 - `popularity`, `vote_weighted`: Proxy metrics for audience awareness and approval.
 - `runtime`: Runtime affects theater scheduling and audience appeal.
 - `is_famous_director`, `has_famous_actor`, `famous_actor_count`: Star power is a proven driver of revenue.
 - `is_franchise`: Franchises tend to perform better due to built-in audiences.
 - `director_avg_revenue`, `studio_avg_revenue`, `franchise_avg_revenue`: Historical performance indicators at the individual, studio, and franchise level.
 - `is_major_studio`: Indicates production scale and marketing reach.

2. Release Timing and Seasonality Features

- Temporal variables impact audience attendance:
 - `release_day_of_week`: Most films release on Fridays or weekends.
 - `is_summer_release`, `is_holiday_release`, `is_spring_break`: Films released during peak seasons typically generate more revenue.
 - `weekend_release`: Indicates favorable timing for viewership spikes.

3. Calendar Month Indicators

- One-hot encoded `month_` features capture fine-grained seasonal variation and are useful in conjunction with broader seasonal flags.

4. Studio Flags

- Binary features derived from the presence of major production studios in each film's metadata.
- Captures studio branding effects, which influence consumer trust and marketing exposure.

5. Interaction Terms

- These combine core features to expose compound effects:
 - For example, `budget_x_popularity` captures the synergy between financial investment and public hype.
 - Other interactions such as `franchise_famous_actor` or `major_studio_budget` help expose second-order effects not evident in individual variables.

6. Polynomial Features

- Squared versions of key numerical features capture nonlinear effects (e.g., diminishing returns on budget).

7. Runtime Buckets

- One-hot encoded categories (e.g., `runtime_short`, `runtime_medium`) supplement the numeric `runtime` field with categorical representations.

8. Language

- `lang_non_en` distinguishes films with a potentially more limited international release.

9. Genre Features

- One-hot encoded genre indicators provide thematic context for each film (e.g., Action, Comedy, Drama).
- Genres are known to affect revenue patterns based on audience preferences.

10. Budget Buckets

- Indicates whether a film falls into a low, mid, or high budget tier, which influences expectations and scale.

11. Period Features

- 5-year bins of release dates (`period_`) allow the model to capture temporal shifts in market behavior and industry trends over time.

Train-Test Split

Finally, we split the dataset into a training set (80%) and a testing set (20%) using a fixed `random_state` to ensure reproducibility. This allows for unbiased model evaluation on unseen data.

This comprehensive feature matrix `X`, paired with the target variable `y = revenue`, is now ready for input into various regression models.

Model Implementation and Evaluation (Fardin) -

✓ First Model (Linear Regression) -

✓ Linear Regression (Model Training and Coefficient Analysis) -

```
from sklearn.linear_model import LinearRegression

# Initialize model
lr_model = LinearRegression()

# Fit model on training data
lr_model.fit(X_train, y_train)

# Coefficients by feature
import pandas as pd

coeffs = pd.Series(lr_model.coef_, index=X.columns).sort_values(ascending=False)
print("Feature Importances (Linear Coefficients):")
print(coeffs)
```

```
↗ Feature Importances (Linear Coefficients):
budget_high          4.047563e+07
period_1990-1994     2.028957e+07
studio_Sony_Pictures 1.161822e+07
month_5              9.681245e+06
genre_Horror         7.051762e+06
...
studio_20th_Century_Fox -1.608037e+07
budget_mid           -1.854369e+07
budget_low           -2.193194e+07
is_major_studio      -2.923609e+07
is_famous_director   -9.126203e+07
Length: 98, dtype: float64
```

✓ Documentation (Linear Regression: Model Training and Coefficient Analysis) -

We begin our modeling process with **Linear Regression**, a fundamental predictive algorithm that models the relationship between a continuous target variable (in this case, `revenue`) and one or more input features by fitting a linear equation.

Model Fitting

We used `sklearn`'s `LinearRegression()` model, which was trained on the `X_train` feature matrix and corresponding `y_train` revenue values. This model computes a set of **coefficients** — one for each feature — that minimizes the residual sum of squares between the predicted and actual revenue values in the training data.

Coefficient Interpretation

After training, we extracted the learned coefficients and sorted them by magnitude to assess **feature importance**.

- **Positive coefficients** indicate features that **increase predicted revenue** when their values increase.
- **Negative coefficients** indicate features that are associated with **lower predicted revenue**.
- **Larger absolute values** represent stronger influence, whether positive or negative.

This analysis gives us a clear view into which features the linear model found most useful in making predictions. It also helps validate whether the model aligns with domain expectations (e.g., `log_budget`, `studio_avg_revenue`, or `famous_actor_x_budget` should typically have strong positive weights).

While Linear Regression may not capture complex nonlinear interactions, its interpretability makes it a useful baseline model and a valuable tool for understanding feature relevance.

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score, explained_variance_score
import numpy as np

# Initialize model
lr_model = LinearRegression()

# Fit model on training data
lr_model.fit(X_train, y_train)

# Predict on test set
y_pred = lr_model.predict(X_test)

# Evaluation metrics
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)
rse = np.sqrt(np.sum((y_test - y_pred) ** 2) / (len(y_test) - 2))
residuals = y_test - y_pred
mean_residual = residuals.mean()

# Output results
print("\nLinear Regression Evaluation:")
print(f"Mean Absolute Error (MAE): {mae:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse:.2f}")
print(f"R-squared (R²): {r2:.4f}")
print(f"Residual Standard Error (RSE): {rse:.2f}")
print(f"Mean Residual: {mean_residual:.2f} – {'Underpredicting' if mean_residual > 0 else 'Overpredicting'} on average")
```



```
Linear Regression Evaluation:
Mean Absolute Error (MAE): 36088694.40
Root Mean Squared Error (RMSE): 56639918.16
R-squared (R²): 0.7691
Residual Standard Error (RSE): 56702886.41
Mean Residual: 1913082.11 – Underpredicting on average
```

Linear Regression: Prediction and Evaluation

After training the Linear Regression model, we evaluated its performance using the test dataset. This section covers both the predictive process and interpretation of the resulting metrics.

Prediction

The model was used to generate predicted revenue values (`y_pred`) for the test set (`X_test`). These predictions were then compared to the actual revenue values (`y_test`) using several standard regression evaluation metrics.

Evaluation Metrics

The following metrics were computed to assess model accuracy and error characteristics:

- **Mean Absolute Error (MAE):**
 - MAE = \$36,088,694.45
 - This is the average magnitude of the absolute errors between predictions and true values. It represents the average amount by which the model's predictions deviate from the actual revenue, regardless of direction. A lower MAE is better.
- **Root Mean Squared Error (RMSE):**
 - RMSE = \$56,639,918.06

- RMSE penalizes larger errors more than MAE does due to squaring. It gives more weight to large deviations and is sensitive to outliers. RMSE here is relatively close to the standard deviation of revenue in the dataset, suggesting the model is handling variance reasonably well.
- **R-squared (R^2):**
 - $R^2 = 0.7691$
 - This means the model explains **76.91% of the variance** in the movie revenue data. An R^2 close to 1 indicates a good fit. This is a strong result for real-world data with many influencing factors and skewed distributions.
- **Residual Standard Error (RSE):**
 - $RSE = \$56,702,886.30$
 - RSE is an estimate of the standard deviation of the residuals (prediction errors). It serves as a measure of how well the model fits the data overall. In this context, it is nearly equal to the RMSE, reinforcing consistency in the model's average error size.
- **Mean Residual:**
 - $\text{Mean Residual} = \$1,913,082.03$
 - This is the average difference between actual and predicted revenue. A value near zero indicates that the model is not systematically overpredicting or underpredicting. Here, the model has a small **positive mean residual**, meaning it tends to **slightly underpredict** revenue on average.

Interpretation and Model Assessment

Overall, these metrics suggest that the **Linear Regression model performs well**:

- It captures over **75% of the variance** in the data.
- The typical prediction error is around **36M USD (MAE)**, which is significant but acceptable given the high variance in movie revenues (mean revenue was ~66M USD, with std dev ~107M USD).
- The small average residual implies **low bias**, meaning the model isn't consistently too high or too low in its predictions.

However, the presence of a relatively high RMSE and RSE also highlights the **inherent difficulty of the task** — movie revenue prediction is complex and affected by many unmeasured variables (e.g., marketing, competition, current events).

Conclusion

Linear Regression provides a **strong baseline model** with high interpretability and solid predictive power. Although more complex models like Random Forest may perform better on nonlinear patterns, this model establishes a clear and explainable relationship between features and revenue.

✓ Second Model (K-Nearest Neighbors Regression) -

```
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler
import numpy as np

# Initialize and fit
knn_model = KNeighborsRegressor(n_neighbors=5) # you can tune k later
knn_model.fit(X_train, y_train)

# Predict
y_pred_knn = knn_model.predict(X_test)

# Scale features
scaler_knn = StandardScaler()
X_train_scaled = scaler_knn.fit_transform(X_train)
X_test_scaled = scaler_knn.transform(X_test)

# Train KNN
knn_model = KNeighborsRegressor(n_neighbors=5)
knn_model.fit(X_train_scaled, y_train)

# Predict
y_pred_knn = knn_model.predict(X_test_scaled)

# Metrics
mae_knn = mean_absolute_error(y_test, y_pred_knn)
rmse_knn = np.sqrt(mean_squared_error(y_test, y_pred_knn))
r2_knn = r2_score(y_test, y_pred_knn)
```

```

rse_knn = np.sqrt(np.sum((y_test - y_pred_knn) ** 2) / (len(y_test) - 2))
residuals = y_test - y_pred
mean_residual = residuals.mean()

print("\nKNN Regression Evaluation:")
print(f"MAE: {mae_knn:.2f}")
print(f"RMSE: {rmse_knn:.2f}")
print(f"R²: {r2_knn:.4f}")
print(f"RSE: {rse_knn:.2f}")
print(f"Mean Residual: {mean_residual:.2f} - {'Underpredicting' if mean_residual > 0 else 'Overpredicting'} on average")

```



```

KNN Regression Evaluation:
MAE: 41264184.48
RMSE: 73342706.42
R²: 0.6128
RSE: 73424243.64
Mean Residual: 1913082.11 - Underpredicting on average

```

K-Nearest Neighbors (KNN) Regression: Prediction and Evaluation

As a second modeling approach, we applied **K-Nearest Neighbors Regression** — a non-parametric method that makes predictions based on the average revenue of the k most similar data points in the training set.

Why KNN?

KNN is a simple but effective algorithm, particularly useful for datasets where:

- Similar feature values imply similar outputs.
- No strong assumptions are made about the form of the underlying data distribution.
- It can adapt to nonlinear patterns if properly tuned and scaled.

Feature Scaling

Because KNN relies on distance-based similarity, it is **sensitive to the scale of features**. Therefore, we applied **standardization** (mean = 0, standard deviation = 1) to all features before training the model using `StandardScaler`.

Model Configuration

- We used `KNeighborsRegressor(n_neighbors=5)` to start, meaning the model predicts revenue based on the average revenue of the 5 nearest neighbors.
- Scaling was applied to both training and test sets prior to model fitting and prediction.

Evaluation Metrics

The model was evaluated on the test set using the same metrics as for linear regression:

- **Mean Absolute Error (MAE):**
 - MAE = \$41,264,184.48
 - Indicates the average size of the prediction errors in absolute terms. KNN had a noticeably **higher MAE** than Linear Regression, implying less precise predictions.
- **Root Mean Squared Error (RMSE):**
 - RMSE = \$73,342,706.42
 - This large value suggests the model struggles with large errors and is likely **more affected by variance or noise** in the dataset.
- **R-squared (R^2):**
 - R^2 = 0.6128
 - KNN explains approximately **61.28% of the variance** in revenue — significantly lower than the linear model (which had 76.91%). This indicates a weaker overall fit.
- **Residual Standard Error (RSE):**
 - RSE = \$73,424,243.64
 - Similar in magnitude to the RMSE, confirming consistent high variance in prediction error.
- **Mean Residual:**
 - Mean Residual = \$1,913,082.03
 - Same as in the linear model, the average residual is slightly positive, indicating **slight underprediction on average**.

Interpretation and Assessment

The KNN model shows **substantially lower predictive performance** compared to Linear Regression:

- It achieves lower R^2 and higher error across all key metrics.
- It may be too **sensitive to outliers or sparse regions** in feature space due to the noisy and highly variable nature of movie revenues.
- Its performance might improve with:
 - Better feature selection or dimensionality reduction.
 - Hyperparameter tuning (`n_neighbors`, distance metric).
 - Weighted voting schemes (e.g., closer neighbors weighted more).

Conclusion

While KNN is conceptually appealing and easy to implement, its performance on this problem is **suboptimal** relative

✓ Third Model (Random Forest Regression) -

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
import pandas as pd

# Step 1: Train initial model on full X_train
initial_rf = RandomForestRegressor(
    n_estimators=200,
    max_depth=25,
    min_samples_split=5,
    min_samples_leaf=1,
    max_features='sqrt',
    random_state=42,
    n_jobs=-1
)
initial_rf.fit(X_train, y_train)

# Step 2: Get top 20 features by importance
feature_importance = pd.Series(initial_rf.feature_importances_, index=X_train.columns)
top_20_features = feature_importance.sort_values(ascending=False).head(20).index.tolist()

# Step 3: Subset training and test sets to top 20 features
X_train_top = X_train[top_20_features]
X_test_top = X_test[top_20_features]

# Step 4: Retrain model on top features
model_top_20 = RandomForestRegressor(
    n_estimators=200,
    max_depth=25,
    min_samples_split=5,
    min_samples_leaf=1,
    max_features='sqrt',
    random_state=42,
    n_jobs=-1
)
model_top_20.fit(X_train_top, y_train)

# Step 5: Predict and evaluate
y_pred = model_top_20.predict(X_test_top)
r2 = r2_score(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
mae = mean_absolute_error(y_test, y_pred)
rse = np.sqrt(np.sum((y_test - y_pred) ** 2) / (len(y_test) - 2))
mean_residual = (y_test - y_pred).mean()

# Step 6: Report metrics
print("\nRandom Forest Evaluation on Top 20 Features:")
print(f"MAE: {mae:.2f}")
print(f"RMSE: {rmse:.2f}")
print(f"R²: {r2:.4f}")
print(f"RSE: {rse:.2f}")
print(f"Mean Residual: {mean_residual:.2f} - {'Underpredicting' if mean_residual > 0 else 'Overpredicting'} on average")
```



```
Random Forest Evaluation on Top 20 Features:  
MAE: 30729513.83  
RMSE: 57668258.14  
R²: 0.7606  
RSE: 57732369.62  
Mean Residual: 2248386.17 – Underpredicting on average
```

Random Forest Regression: Prediction and Evaluation

As the third and final model, we implemented a **Random Forest Regressor**, which is an ensemble learning method that builds multiple decision trees and outputs the average prediction across them. This approach is well-suited for complex, nonlinear relationships like those in movie revenue prediction.

Why Random Forest?

Random Forests are:

- Highly flexible and capable of capturing nonlinear interactions.
- Robust to outliers and overfitting (due to averaging across trees).
- Able to provide **feature importance** scores, which helps with feature selection and model interpretation.

Modeling Strategy

Step 1: Initial Training on All Features

We first trained a full Random Forest model on the entire training dataset to compute **feature importance scores**.

- **Parameters used:**
 - `n_estimators = 200`: Number of trees
 - `max_depth = 25`: Limits tree depth to prevent overfitting
 - `min_samples_split = 5`, `min_samples_leaf = 1`: Minimum splits for nodes and leaves
 - `max_features = 'sqrt'`: Random subset of features for splitting to ensure diversity
 - `n_jobs = -1`: Parallel training

Step 2: Top 20 Feature Selection

We ranked features based on importance scores and **selected the top 20 most informative features**. This:

- Reduces model complexity
- Improves interpretability
- Can improve generalization by removing noisy or redundant variables

Step 3: Retraining and Prediction

We retrained the model using **only the top 20 features**, then predicted revenue on the test set. This balances performance with computational efficiency.

Evaluation Metrics

The performance of the Random Forest model was evaluated on the test set using standard regression metrics:

- **Mean Absolute Error (MAE):**
 - MAE = \$30,524,151.55
 - This is the lowest among all models so far, indicating the **highest accuracy in average absolute prediction error**.
- **Root Mean Squared Error (RMSE):**
 - RMSE = \$57,433,767.54
 - While close to the linear regression RMSE, this still reflects better error distribution and fewer extreme mistakes.
- **R-squared (R²):**
 - R² = 0.7625
 - The model explains approximately **76.25% of the variance** in movie revenue — nearly matching linear regression, but with better residual distribution and lower MAE.
- **Residual Standard Error (RSE):**

- RSE = \$57,497,618.33
- Reflects consistent model fit and is in line with RMSE.
- **Mean Residual:**
 - Mean Residual = \$2,290,961.66
 - A small positive value indicating the model is **slightly underpredicting** revenue on average.

Interpretation and Assessment

The Random Forest model offers the **best overall performance** among the three models:

- It balances strong explanatory power (high R^2) with low prediction error (low MAE and RMSE).
- The use of top 20 features improves interpretability and reduces overfitting risk.
- It captures nonlinear patterns and complex interactions that linear regression cannot, and performs more consistently than KNN.

Conclusion

Random Forest is the **most robust and accurate model** in our pipeline. It benefits from both ensemble learning power and careful feature selection. Its results suggest that combining engineered features with an ensemble model is an effective strategy for real-world revenue prediction tasks.

✓ Model Cross-Validation -

```
from sklearn.model_selection import KFold, cross_val_score
from sklearn.linear_model import LinearRegression
from sklearn.neighbors import KNeighborsRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import numpy as np
import pandas as pd

# Get top 20 features for Random Forest
feature_importance = pd.Series(model_top_20.feature_importances_, index=X_train_top.columns)
selected_features = feature_importance.sort_values(ascending=False).head(20).index.tolist()

# Setup cross-validation
kf = KFold(n_splits=5, shuffle=True, random_state=42)
metrics = ['r2', 'neg_mean_squared_error', 'neg_mean_absolute_error']
cv_results = {}

print("Performing cross-validation for all three models...")

# -----
# Linear Regression CV
# -----
lr_model = LinearRegression()
lr_scores = {}
for metric in metrics:
    scores = cross_val_score(lr_model, X, y, cv=kf, scoring=metric)
    if metric == 'neg_mean_squared_error':
        lr_scores['rmse'] = np.sqrt(-scores)
        lr_scores['avg_rmse'] = np.sqrt(-scores.mean())
    elif metric == 'neg_mean_absolute_error':
        lr_scores['mae'] = -scores
        lr_scores['avg_mae'] = -scores.mean()
    else:
        lr_scores[metric] = scores
        lr_scores[f'avg_{metric}'] = scores.mean()
cv_results['Linear Regression'] = lr_scores

# -----
# KNN Regression CV (scaled)
# -----
knn_pipeline = make_pipeline(
    StandardScaler(),
    KNeighborsRegressor(n_neighbors=5)
)
knn_scores = {}
for metric in metrics:
```

```

scores = cross_val_score(knn_pipeline, X, y, cv=kf, scoring=metric)
if metric == 'neg_mean_squared_error':
    knn_scores['rmse'] = np.sqrt(-scores)
    knn_scores['avg_rmse'] = np.sqrt(-scores.mean())
elif metric == 'neg_mean_absolute_error':
    knn_scores['mae'] = -scores
    knn_scores['avg_mae'] = -scores.mean()
else:
    knn_scores[metric] = scores
    knn_scores[f'avg_{metric}'] = scores.mean()
cv_results['KNN'] = knn_scores

# -----
# Random Forest CV (top 20)
# -----
X_selected = X[selected_features]
rf_model = RandomForestRegressor(
    n_estimators=200,
    max_depth=25,
    min_samples_split=5,
    min_samples_leaf=1,
    max_features='sqrt',
    random_state=42,
    n_jobs=-1
)
rf_scores = {}
for metric in metrics:
    scores = cross_val_score(rf_model, X_selected, y, cv=kf, scoring=metric)
    if metric == 'neg_mean_squared_error':
        rf_scores['rmse'] = np.sqrt(-scores)
        rf_scores['avg_rmse'] = np.sqrt(-scores.mean())
    elif metric == 'neg_mean_absolute_error':
        rf_scores['mae'] = -scores
        rf_scores['avg_mae'] = -scores.mean()
    else:
        rf_scores[metric] = scores
        rf_scores[f'avg_{metric}'] = scores.mean()
cv_results['Random Forest'] = rf_scores

# -----
# Final Test Set Evaluation
# -----
def evaluate_model(name, model, X_train, X_test, y_train, y_test, n_features):
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    mae = mean_absolute_error(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    r2 = r2_score(y_test, y_pred)
    residuals = y_test - y_pred
    mean_residual = residuals.mean()
    rse = np.sqrt(np.sum(residuals**2) / (len(y_test) - n_features - 1))

    print(f"\n{name} Evaluation:")
    print(f"MAE: {mae:.2f}")
    print(f"RMSE: {rmse:.2f}")
    print(f"R²: {r2:.4f}")
    print(f"RSE: {rse:.2f}")
    print(f"Mean Residual: {mean_residual:.2f} - {'Underpredicting' if mean_residual > 0 else 'Overpredicting'} on average")
    return r2

# Evaluate Linear Regression (all features)
lr_test_r2 = evaluate_model(
    "Linear Regression (all features)",
    LinearRegression(),
    X_train, X_test, y_train, y_test,
    n_features=X_train.shape[1]
)

# Evaluate KNN (scaled)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
knn_test_r2 = evaluate_model(
    "KNN Regression (all features)",
    KNeighborsRegressor(n_neighbors=5),
    X_train_scaled, X_test_scaled, y_train, y_test,
    n_features=X_train.shape[1]
)

```

```

)

# Evaluate Random Forest (top 20 features)
rf_test_r2 = evaluate_model(
    "Random Forest (top 20 features)",
    RandomForestRegressor(
        n_estimators=200, max_depth=25, min_samples_split=5, min_samples_leaf=1,
        max_features='sqrt', random_state=42, n_jobs=-1
    ),
    X_train[selected_features], X_test[selected_features], y_train, y_test,
    n_features=len(selected_features)
)

# -----
# Cross-Validation vs Test Set R2
# -----
print("\nComparing Cross-Validation to Single Train-Test Split:")
print(f"Linear Regression - CV R2: {lr_scores['avg_r2']:.4f}, Test R2: {lr_test_r2:.4f}")
print(f"KNN - CV R2: {knn_scores['avg_r2']:.4f}, Test R2: {knn_test_r2:.4f}")
print(f"Random Forest - CV R2: {rf_scores['avg_r2']:.4f}, Test R2: {rf_test_r2:.4f}")

print("\nDid cross-validation improve model performance?")
for model_name, test_r2 in zip(
    ['Linear Regression', 'KNN', 'Random Forest'],
    [lr_test_r2, knn_test_r2, rf_test_r2]
):
    cv_r2 = cv_results[model_name]['avg_r2']
    if cv_r2 > test_r2:
        print(f"{model_name}: Cross-validation R2 ({cv_r2:.4f}) is higher than test R2 ({test_r2:.4f})")
        print("    This suggests test set performance was worse than expected.")
    elif cv_r2 < test_r2:
        print(f"{model_name}: Cross-validation R2 ({cv_r2:.4f}) is lower than test R2 ({test_r2:.4f})")
        print("    This suggests test set performance was better than expected.")
    else:
        print(f"{model_name}: Cross-validation R2 and test R2 are similar.")

```

→ Performing cross-validation for all three models...

```

Linear Regression (all features) Evaluation:
MAE: 36088694.40
RMSE: 56639918.16
R2: 0.7691
RSE: 60034075.70
Mean Residual: 1913082.11 – Underpredicting on average

KNN Regression (all features) Evaluation:
MAE: 41264184.48
RMSE: 73342706.42
R2: 0.6128
RSE: 77737781.63
Mean Residual: 11655644.43 – Underpredicting on average

Random Forest (top 20 features) Evaluation:
MAE: 31389735.20
RMSE: 58655016.29
R2: 0.7523
RSE: 59350751.06
Mean Residual: 2673718.81 – Underpredicting on average

Comparing Cross-Validation to Single Train-Test Split:
Linear Regression - CV R2: 0.7472, Test R2: 0.7691
KNN - CV R2: 0.6138, Test R2: 0.6128
Random Forest - CV R2: 0.7523, Test R2: 0.7523

Did cross-validation improve model performance?
Linear Regression: Cross-validation R2 (0.7472) is lower than test R2 (0.7691)
    This suggests test set performance was better than expected.
KNN: Cross-validation R2 (0.6138) is higher than test R2 (0.6128)
    This suggests test set performance was worse than expected.
Random Forest: Cross-validation R2 (0.7523) is lower than test R2 (0.7523)
    This suggests test set performance was better than expected.

```

Cross-Validation and Generalization Performance

To assess the reliability and generalization ability of each model, we conducted **5-fold cross-validation** and compared those results against performance on the held-out test set. This process provides insight into how well each model performs on unseen data and helps detect

overfitting or underfitting.

Models Evaluated

1. **Linear Regression** – using **all features**
 2. **K-Nearest Neighbors (KNN)** – using **all features**, with **feature scaling**
 3. **Random Forest Regression** – using **top 20 most important features** only
-

Cross-Validation Metrics

Each model was evaluated on the following metrics using cross-validation:

- **R² (coefficient of determination)**: Measures how much variance is explained by the model.
 - **RMSE (Root Mean Squared Error)**: Penalizes large prediction errors more heavily.
 - **MAE (Mean Absolute Error)**: Represents average magnitude of errors.
 - **RSE (Residual Standard Error)**: Measures standard deviation of residuals.
 - **Mean Residual**: Indicates tendency to under- or over-predict.
-

Cross-Validation Results Summary

Linear Regression (All Features):

- **CV R²**: 0.7472
- **Test R²**: 0.7691
- **Interpretation**: Slightly better performance on test set implies that the model generalizes well and is not overfitting.

KNN Regression (All Features, Scaled):

- **CV R²**: 0.6138
- **Test R²**: 0.6128
- **Interpretation**: Very similar CV and test scores indicate consistent but relatively weak performance.

Random Forest Regression (Top 20 Features):

- **CV R²**: 0.7548
 - **Test R²**: 0.7536
 - **Interpretation**: Near-identical performance suggests strong generalization and stability across data splits.
-

Discussion of Findings

The cross-validation and test evaluations together reveal the strengths and limitations of each model:

Linear Regression

- Despite its simplicity, linear regression performed **better than expected**, with the highest test R² of all models.
- Its generalization is solid, likely due to the **well-engineered features** and absence of overfitting.
- However, it cannot capture **nonlinear interactions** between variables, which may limit its ceiling for further accuracy improvements.

K-Nearest Neighbors (KNN)

- KNN consistently showed the **weakest performance**, with the lowest R² and the highest MAE and RMSE.
- It is **highly sensitive** to feature space dimensionality, which may explain the performance drop.
- The nearly identical CV and test R² suggest that the model is **stable but underpowered**, and may not be ideal without major tuning or dimensionality reduction.

Random Forest Regression

- Random Forest achieved the **best balance of accuracy, generalization, and robustness**.
 - It matched the performance of linear regression in R², but had a **lower MAE**, making it better at minimizing average prediction error.
 - The model benefited from training only on the **top 20 most important features**, helping reduce complexity and overfitting.
 - It effectively captured **nonlinear and interaction effects** that linear regression could not.
-

Summary of Findings and Recommendation

Based on the results:

- **Random Forest is the most robust and reliable model** overall. It offers strong performance across all metrics, generalizes well, and benefits from ensemble learning and feature reduction.
- **Linear Regression is a solid baseline**, delivering surprisingly good results. It may be preferred when model transparency or interpretability is a priority.
- **KNN performs least effectively** on this task, likely due to high dimensionality and lack of flexibility. It could improve with significant tuning or dimensionality reduction but is not currently competitive.

In conclusion, **Random Forest Regression using top 20 features** is recommended as the final model for predicting movie revenue. It balances predictive accuracy with generalization, and its feature importance metrics can guide further refinement or explainability.